# Ionic + Angular
# Mi práctica con Sass

Práctica del curso · 23 de octubre de 2026

He preparado una página Home con un título rojo, en mayúsculas y centrado; un botón azul a la derecha; y una imagen centrada con borde verde de 3 px. En esta memoria explico el procedimiento, la compilación de Sass y los cambios de configuración.

## Mi procedimiento

**1.** He descomprimido los dos ZIP del curso y he conservado sus ejemplos de fundamentos de Sass y arquitectura 7-1.
**2.** He preparado el proyecto ionic-sass-home con Ionic Angular y una página Home standalone.
**3.** He vinculado la plantilla HTML y el archivo home.page.sass; después he aplicado variables, un mixin y anidamiento.
**4.** He iniciado el servidor con ionic serve, mediante npm start, y he comprobado el resultado en http://localhost:8100/home.




Captura 1. He abierto Home en localhost y he comprobado el título, el botón y la imagen con los estilos de la práctica.

Para arrancar mi proyecto utilizo estos comandos desde su carpeta:

```
npm install
npm start
# npm start ejecuta ionic serve
```

## Solución · 1. ¿El componente usa Sass por defecto?

**Sí, en la configuración habitual de Ionic con Angular se utiliza Sass mediante la sintaxis SCSS.** Al generar un proyecto con el CLI oficial, la plantilla suele crear los estilos con extensión .scss:

```
ionic start nombre blank --type=angular
```

Distingo el comportamiento de esa plantilla de mi entrega: mi proyecto ya está preparado y usa .sass. No he ejecutado el comando anterior para sobrescribirlo; lo incluyo como referencia del flujo habitual de creación.

## Configuración predeterminada del starter

En angular.json, dentro del proyecto correspondiente, identifico la opción de generación de componentes. En la configuración SCSS habitual tiene este valor:

```
"schematics": {
  "@schematics/angular:component": {
    "style": "scss"
  }
}
```

La página Home autogenerada incluye home.page.scss y lo vincula en el decorador @Component. Por ejemplo:

```
@Component({
  selector: 'app-home',
  templateUrl: 'home.page.html',
  styleUrls: ['home.page.scss']
})
```

También puedo usar styleUrl, en singular, cuando vinculo un solo archivo en versiones recientes de Angular; no es una opción exclusiva de los componentes standalone.

## Cómo distingo .scss y .sass

Entiendo SCSS como una sintaxis de Sass compatible con CSS, con llaves y puntos y coma. Si quiero la sintaxis indentada original, utilizo .sass, sin llaves ni puntos y coma, y lo configuro expresamente. En mi práctica he elegido esa segunda opción: **home.page.sass**.

No confundo, por tanto, el lenguaje Sass con una extensión concreta: tanto .scss como .sass se procesan con el compilador de Sass. La extensión .scss es la convención habitual del starter de Ionic Angular.

## Solución · 2. ¿Se compila solo con ionic serve?

**Sí. Al aplicar Sass, la compilación es automática mientras ejecuto ionic serve; no necesito npm run watch:sass.** Ionic delega en el servidor de desarrollo de Angular CLI. En mi configuración ejecuta el target app:serve, equivalente al flujo de ng serve.

## El pipeline de compilación que utilizo

El sistema de construcción de Angular integra Dart Sass para convertir mis estilos en CSS. Según la versión y el builder, Angular puede usar esbuild o Webpack. **En mi entrega utilizo esbuild** para la construcción y Vite para el servidor de desarrollo.

Cuando inicio ionic serve, el proceso que sigo es este:
**1.** Angular detecta los archivos .scss o .sass vinculados a los componentes y sus importaciones.
**2.** Dart Sass transforma el código en CSS; durante el desarrollo Angular lo sirve al navegador sin que yo genere un CSS manual aparte.
**3.** Angular aplica por defecto ViewEncapsulation.Emulated a los estilos de mi componente, acotando sus selectores.
**4.** El watcher permanece activo: cuando guardo home.page.sass, Angular recompila y actualiza los estilos en el navegador.

En el caso del starter SCSS, ocurre lo mismo al guardar home.page.scss. En mi proyecto el archivo se llama home.page.sass. El Shadow DOM interno de algunos elementos Ionic es independiente de esta encapsulación de Angular.

## Por qué no ejecuto watch:sass en Ionic

En los dos ejemplos HTML del curso sí utilizo npm run watch:sass, porque tienen un compilador independiente que escribe dist/css/main.css. En mi aplicación Ionic aprovecho el watcher integrado de Angular. Si detengo el servidor, guardar el archivo ya no provoca esa recompilación automática.




Captura 2. He pulsado «Probar botón» y el contador muestra «Pulsaciones: 1». Así compruebo la interacción de Home en el servidor local.

## Qué bundle utilizo y dónde cambio SCSS a Sass

He utilizado **Ionic Angular 8 con Angular 20**, TypeScript y componentes standalone. En angular.json he configurado @angular/build:application, que usa esbuild para crear el bundle de producción. Para desarrollo utilizo @angular/build:dev-server, basado en Vite. Dart Sass se encarga del preprocesado de estilos dentro de ese flujo.

```
"build": {
  "builder": "@angular/build:application"
},
"serve": {
  "builder": "@angular/build:dev-server"
}
```

Para generar la salida de producción ejecuto npm run build. Obtengo los archivos web en www/browser. No necesito desplegar la práctica en Vercel.

## Mi configuración en angular.json

Dentro de projects.app he cambiado la generación de futuros componentes a sass. También he indicado el lenguaje de estilos inline y la ruta de mi hoja global:

```
"schematics": {
  "@schematics/angular:component": { "style": "sass" }
}

// Dentro de architect.build.options:
"inlineStyleLanguage": "sass",
"styles": ["src/global.sass"]
```

## La vinculación de mi página Home

```
@Component({
  selector: 'app-home',
  standalone: true,
  imports: [IonContent, IonButton],
  templateUrl: './home.page.html',
  styleUrl: './home.page.sass'
})
```

Para migrar un proyecto existente, además de cambiar estas opciones, renombro los archivos .scss a .sass que quiero convertir, adapto su contenido a la sintaxis indentada y actualizo las referencias. La configuración no convierte automáticamente los archivos existentes. He aplicado lo mismo al parcial _tokens.sass.

## Mis comentarios relevantes del código

He centralizado los colores en src/theme/_tokens.sass y he creado un mixin para reutilizar el centrado de elementos de bloque:

```
$title-red: #c62828
$button-blue: #1565c0
$border-green: #16803c

@mixin centered($width)
  display: block
  width: $width
  margin-inline: auto
```

En home.page.sass he aplicado esos valores y he agrupado los estilos mediante anidamiento. El selector & me permite generar clases como .home__title:

```
@use '../../theme/tokens' as theme

.home
  // He centrado el título y lo he puesto en rojo y mayúsculas.
  &__title
    color: theme.$title-red
    text-transform: uppercase
    text-align: center

  // He llevado el botón al extremo derecho con Flexbox.
  &__actions
    display: flex
    justify-content: flex-end

  // He usado la variable CSS de Ionic para el fondo interno.
  &__button
    --background: #{theme.$button-blue}

  // He centrado la imagen y añadido el borde verde de 3 px.
  &__image
    @include theme.centered(100%)
    max-width: 720px
    height: auto
    border: 3px solid theme.$border-green
```

En ion-button he utilizado --background con interpolación Sass para personalizar el botón interno de Ionic a través de sus propiedades CSS. En la imagen he usado width: 100%, max-width y height: auto para conservar las proporciones y adaptarla a pantallas pequeñas.

## Comprobación final en localhost

He comprobado la página a 390 px de ancho para revisar cómo se adapta en móvil. Mantengo el título centrado, el botón a la derecha y la imagen dentro del ancho disponible, sin desbordamiento horizontal.




Captura 3. He revisado Home en una pantalla móvil de 390 px. La imagen conserva sus proporciones y el borde verde.

## Mis comprobaciones

He ejecutado npm run build correctamente. El bundle inicial tiene 485,37 kB y una transferencia estimada de 134,12 kB. He comprobado que la imagen local carga y que el botón incrementa el contador tanto con el ratón como con la tecla Enter. También he compilado los dos ejemplos del curso.

## Mi entrega

He incluido el proyecto, los ejemplos descomprimidos, las capturas y esta memoria. El repositorio de destino es github.com/lucaslopez72/estilos-y-Scss. He publicado los archivos del proyecto en la rama main, conservando su estructura de carpetas, para que puedan consultarse y descargarse desde GitHub.